



Contents lists available at ScienceDirect

Digital Communications and Networks

journal homepage: www.keaipublishing.com/dcan

Fast UAV path planning in urban environments based on three-step experience buffer sampling DDPG

Shasha Tian^{a,b}, Yuanxiang Li^c, Xiao Zhang^{a,b,*}, Lu Zheng^{a,b}, Linhui Cheng^a, Wei She^d, Wei Xie^{a,b}^a College of Computer Science, South-Central Minzu University, Wuhan, 430074, China^b Hubei Provincial Engineering Research Center for Intelligent Management of Manufacturing Enterprises, Wuhan, 430074, China^c School of Computer Science, Wuhan University, Wuhan, 430072, China^d School of Mathematics & Statistics, South-Central Minzu University, Wuhan, 430074, China

ARTICLE INFO

Keywords:

Unmanned aerial vehicle

Path planning

Deep deterministic policy gradient

Three-step experience buffer

Particle swarm optimization

ABSTRACT

The path planning of Unmanned Aerial Vehicle (UAV) is a critical issue in emergency communication and rescue operations, especially in adversarial urban environments. Due to the continuity of the flying space, complex building obstacles, and the aircraft's high dynamics, traditional algorithms cannot find the optimal collision-free flying path between the UAV station and the destination. Accordingly, in this paper, we study the fast UAV path planning problem in a 3D urban environment from a source point to a target point and propose a Three-Step Experience Buffer Deep Deterministic Policy Gradient (TSEB-DDPG) algorithm. We first build the 3D model of a complex urban environment with buildings and project the 3D building surface into many 2D geometric shapes. After transformation, we propose the Hierarchical Learning Particle Swarm Optimization (HL-PSO) to obtain the empirical path. Then, to ensure the accuracy of the obtained paths, the empirical path, the collision information and fast transition information are stored in the three experience buffers of the TSEB-DDPG algorithm as dynamic guidance information. The sampling ratio of each buffer is dynamically adapted to the training stages. Moreover, we designed a reward mechanism to improve the convergence speed of the DDPG algorithm for UAV path planning. The proposed TSEB-DDPG algorithm has also been compared to three widely used competitors experimentally, and the results show that the TSEB-DDPG algorithm can archive the fastest convergence speed and the highest accuracy. We also conduct experiments in real scenarios and compare the real path planning obtained by the HL-PSO algorithm, DDPG algorithm, and TSEB-DDPG algorithm. The results show that the TSEB-DDPG algorithm can archive almost the best in terms of accuracy, the average time of actual path planning, and the success rate.

1. Introduction

Recent years have witnessed many military and commercial applications with Unmanned Aerial Vehicle (UAV), such as emergency communication and rescue operation in urban areas.

UAV path planning is a critical problem of the rescue operation, especially in adversarial urban environments, which is generally formulated as a UAV planning a collision-free optimal or suboptimal safe path in the space with obstacles [1]. However, to conduct an emergency communication task or rescue operation in a 3D urban environment, a UAV usually encounters various obstacles, such as buildings and electrical wires, which may affect the UAV's safety.

Moreover, the convergence speed of the UAV path planning algorithm is highly required for emergency tasks in adversarial urban environments. Due to the continuity of the flying space, complex building obstacles, the high dynamics of the aircraft, and rapid response requirements, it is very challenging to design a fast path planning scheme that can find the optimal collision-free flying path between the UAV station and destination.

Researchers commonly use several methods to solve UAV path planning problems, including traditional path planning methods, swarm intelligence algorithms, and deep reinforcement learning algorithms.

In traditional path planning methods, the artificial potential field method [2], cell decomposition method [3,4], and dynamic window

* Corresponding author. College of Computer Science, South-Central Minzu University, Wuhan, 430074, China.

E-mail addresses: shashatian77@mail.scuec.edu.cn (S. Tian), yxli@whu.edu.cn (Y. Li), xiao.zhang@my.cityu.edu.hk (X. Zhang), lu2008@mail.scuec.edu.cn (L. Zheng), 44393390@qq.com (L. Cheng), wshe@mail.scuec.edu.cn (W. She), xiewei962@qq.com (W. Xie).

<https://doi.org/10.1016/j.dcan.2023.02.016>

Received 31 July 2022; Received in revised form 6 January 2023; Accepted 26 February 2023

Available online 9 March 2023

2352-8648/© 2023 Chongqing University of Posts and Telecommunications. Publishing Services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

method [5] are the most representatives. The artificial potential field method has the advantages of simple structure and easy implementation. However, this method is mainly used to avoid dynamic obstacles in local search. The UAV could fall into the local optimum due to the unreasonable potential field equation when faced with a relatively complex dynamic environment. The cell decomposition method is mainly used for the UAV path planning of fully covering a simple and static environment. The dynamic window method transforms position control into speed control and is used mainly in local path planning.

The swarm intelligence algorithm is inspired by natural phenomena and living laws [6,7]. It can transform the optimal path search problem into a function optimization problem. The swarm intelligence algorithm can self-learn and self-update, which is useful for solving multi-peak optimization problems. Therefore, it is well suited to deal with path planning problems in complex environments. Particle Swarm Optimization (PSO) is a classical and widely used swarm intelligence algorithm. Compared to the other swarm intelligence algorithms, the PSO algorithm is easy to implement and has strong scalability, which has been shown to efficiently solve various benchmark optimization problems with low running time [8].

Some researchers have used the PSO algorithm and its variants to solve the UAV path planning problem and have achieved favourable results. Sun designed a velocity observer to estimate the unmeasurable speed, investigated a path planning method using improved PSO combined with Dijkstra's algorithm, and obtained a fast desired optimal motion path for obstacle avoidance [9]. Shao proposed an improved PSO, which uses a chaos-based logistic map to improve the initial particle distribution and designs adaptive linear-varying constant acceleration coefficients and maximum velocity [10]. The PSO algorithm works well for static global path planning, but the fitness function of the PSO algorithm should be problem-specific. Its adaptability is limited in practice. Moreover, in a dynamic environment, the UAV needs to change its moving path according to the fast situation, which is beyond the capabilities of the PSO algorithm. The PSO algorithm cannot respond quickly to path planning requests in complex and variable urban relief environments.

DDPG is a deep Reinforcement Learning (RL) algorithm for solving continuous control problems. Since it is consistent with the continuous control idea of UAV path planning, it has been applied to UAV path planning. If the model of the UAV path planning algorithm based on the RL algorithm is trained accurately, the response speed of its path planning can be greatly improved compared to the swarm intelligence algorithm. Now many scholars have studied this area. Dong proposes a path planning method based on an improved DDPG reinforcement learning algorithm, which uses a small amount of prior knowledge to accelerate the training of deep reinforcement learning, reduce the number of trials and errors, and adopt an adaptive exploration method based on the ϵ -greedy algorithm [11]. Tu presents a reinforcement learning algorithm to implement UAV path planning and obstacle avoidance [12]. However, the DDPG algorithm is an unsupervised self-learning method. In complex environments and with limited training time, the accuracy of the DDPG algorithm is not guaranteed due to the random learning direction of the agent.

PSO algorithm is a heuristic optimization method which imitates nature. It is simple to operate and has a high search efficiency. However, it is apt to be precocious in its early stages and slow in its later stages. Moreover, for each specific problem, specific modelling training is required, which is not convenient for transfer learning. Deep reinforcement learning algorithm is a class of autonomous learning algorithms developed recently. Since it is based on a trial-and-error learning algorithm, its convergence rate is slow. However, deep reinforcement learning algorithm also has the clear advantage of directly applying to similar problems once training is completed, and facilitating transfer learning. Therefore, in this paper, we combine the two algorithms and use the PSO algorithm optimized according to the problem to provide the initial model parameters for the DDPG algorithm to solve the UAV path

planning problem quickly.

This paper presents a three-step experience buffer DDPG algorithm (TSEB-DDPG) for efficiently solving the continuous fast UAV path planning problem. In the TSEB-DDPG algorithm, the global optimal path information is obtained by hierarchical learning particle swarm optimization (HL-PSO). The collision information and fast transition information are stored in three experience buffers, respectively. The sampling ratio of each buffer is dynamically adapted to the training stages. The path optimization results of HL-PSO are used to provide initial parameter training for the DDPG algorithm model, which can effectively prevent the DDPG algorithm from exploding in dimension. In the later stage, the trial-and-error learning of DDPG algorithm can effectively avoid the precocious convergence of the HL-PSO algorithm.

Our key novelties and main contributions in this paper are summarized as follows.

- A 3D model of a complex urban environment with buildings is constructed. To simplify the problem we study, a geometric approach is proposed to project the 3D building surface onto a myriad of 2D geometries.
- To efficiently solve the UAV path planning problem, the HL-PSO algorithm is proposed. The modified PSO algorithm selects different learning samples for different classes of particles based on population diversity to ensure the efficient evolution of each type. By incorporating the optimization output of the PSO algorithm into the DDPG algorithm, the accuracy of the DDPG algorithm under finite training time is guaranteed.
- Due to the low efficiency of the experience buffer in the DDPG algorithm, three experience buffers are used in combination with the optimization results of the PSO algorithm. We propose a step-wise sampling strategy that allows each experience to play its role in time. The dynamic experience buffer sampling strategy ensures that the experience information exits from the guidance of the DDPG algorithm promptly and improves the self-learning ability.

In this paper, we combine the advantages of HL-PSO and DDPG algorithms with a specific reward policy and propose a solution policy for a particular UAV path planning problem.

2. Related work

PSO is a very classical swarm intelligence algorithm. Although it can not meet the rapid response of path planning in various urgent and dynamic environments, as an excellent heuristic algorithm verified by numerous scientists, it can still play an important role in the rapid response requirements of path planning. Research into the PSO algorithm has a long history in path planning. The path planning problem can be abstracted as a single objective optimization problem. Researchers have done a lot of work to improve the convergence speed and accuracy of the PSO algorithm [13]. Several attempts have been made to improve the basic PSO algorithm, such as parameter adjustment [14–17], multi-population method [18–20], hybrid algorithm [21–23], and learning model [24,25]. In terms of parameter adjustment, researchers mainly studied inertia weight, social cognitive factors, and individual cognitive factors. These parameters are adjusted as linear or nonlinear functions to effectively balance global and local search ability [14–16]. Nobile proposes the Fuzzy Self-Tuning PSO which exploits Fuzzy Logic to calculate the inertia, cognitive and social factor, and minimum and maximum velocity independently for each particle [17]. In the aspect of multi-population, researchers use the interaction and cooperation of multiple populations to search for optimization jointly. Ye proposed a multi-agent hybrid particle swarm optimization to solve the problem of wastewater treatment network planning [18]. Lan selected good particles to form new particle swarms, which were interactively updated with the original particle swarms to create a hierarchical sorting group optimizer for large-scale optimization [19]. Al-Bahrani proposed to overcome the

zigzagging of the direction of search, the orthogonal PSO which particles are updated using only one guide [20]. Xia proposed a dynamic multi-swarm particle swarm optimization algorithm based on an elite learning strategy, which fully uses the advantages of different subgroups and lets them play different roles in different stages of evolution [21]. Researchers also favour a hybrid algorithm. Combining the advantages of the PSO algorithm and other classical algorithms (such as the genetic algorithm and ant colony algorithm), many hybrid algorithms based on the PSO algorithm have been proposed [22,23]. Some researchers also devoted themselves to improving the learning model of the PSO algorithm and seeking better learning samples and learning strategies for particles. Xia reconstructed the learning model of particle swarm optimization by drawing on human memory and forgetting ability, thus increasing the population's diversity and improving algorithm optimization accuracy [24]. Chen proposed a new biogeographic learning strategy for particle swarm optimization [25]. Most research is to improve the PSO algorithm for general optimization problems. In this paper, the PSO algorithm is enhanced according to the characteristics of UAV path planning to improve the algorithm's accuracy and convergence speed. The empirical path obtained by the improved PSO algorithm is used as the experience of the RL algorithm to improve the accuracy and convergence speed of the UAV path planning algorithm.

Deep reinforcement learning algorithms are often used in decision-making problems [26]. In this paper, UAV global path planning is a continuous control problem, so the DDPG algorithm is selected for research. Lillicrap proposed the DDPG algorithm in 2015 to solve the problem of fast continuous control [27]. A key link of the DDPG algorithm is the playback mechanism of the experience buffer. Therefore, many researchers have focused on this research hot spot. Lin proposed the experience playback mechanism in 1992 and used an experience buffer to store past experiences [28]. Hou used the preferred sampling DDPG algorithm to replace the unified sampling to test the inverted pendulum task. The preferred sampling mechanism reduced the training time and improved the training stability [29]. Li proposed replay double-delay DDPG of multiple experiences to balance random power perturbations in integrated energy systems [30]. Zhang used an asynchronous mode to design an experience buffer data acquisition scheme and proposed an asynchronous intermittent DDPG algorithm to solve the problem of continuous control in a complex computing environment [31]. Bruin rewrote the two experience replay databases using different strategies to ensure uniform experience distribution [32]. Rojanaarpa improved learning stability and training speed by storing only rare and highly irrelevant data in the experience buffer [33]. Xiang combined the data pruning in [34] with the priority sampling in [35] and proposed a comprehensive priority data pruning method. To speed up the training process, the experience buffer is first sampled, and then similar samples

are deleted [36]. Peng replaced the neural network optimizer in DDPG with the RAdam algorithm and combined the curiosity algorithm to improve the success rate and convergence rate. In addition, the authors added preferential experience replay and introduced transfer learning to improve training performance [37]. The existing improvements to the experience buffer mainly focus on increasing the utilization of important samples, but some important samples may be missed. In this paper, different experience buffers are used to store different important samples, and dynamic sampling is carried out in stages to improve the algorithm's accuracy.

3. System model

We consider a UAV path planning problem for rescue operation or delivering emergency communication, where the urban environment is modeled in 3D geometrical space with buildings of different heights, as shown in Fig. 1. Denote the 3D space as U , the number of task points is M , and the j th task point is represented as T_j , $j \in \{1, 2, \dots, M\}$.

We denote the starting point as $S(x_0, y_0, z_0)$, the destination point as $E(x_{M+1}, y_{M+1}, z_{M+1})$, and the UAV station as Q_l , $l \in \{1, 2, \dots, L\}$, where L is the number of stations. The obstacles, such as buildings, are denoted as $B_1, B_2, \dots, B_F$, in which B_i is modeled with a function $f(B_i)$ and F is the number of the buildings. $f(B_i)$ represents the shape information of the building B_i . A UAV needs to fly from a starting point to several task points at different altitudes. The UAV is assumed to be equipped with a sufficient battery of energy for the task. Our objective is to design a fast algorithm to find short collision-free flight paths for UAVs to deliver emergency services.

3.1. Transformation of 3D space geometrical model

In complex urban environments, there are many static obstacles, such as buildings, street lights, and signboards, and many dynamic obstacles, such as crowds and vehicles. Therefore, to ensure the execution efficiency of the algorithm studied in this paper and minimize energy consumption, the 3D urban space is modeled as a multi-layer 2D plane during the UAV mission. We construct a stochastic 3D spatial geometry model of an urban environment and project the 3D path planning task onto several 2D planes at different heights.

We divide 3D urban space into a number N of 2D planes P_i , $i \in \{1, 2, \dots, N\}$. To clearly illustrate the dimensional transformation, we illustrate with an example as shown in Fig. 2.

Fig. 2 (a) shows the randomly generated 3D space of the urban environments with 40 buildings divided into three 2D planes. In the space of the first, second and third layers, the 2D plane motion map of the UAV is shown in Fig. 2 (b), (c) and (d). We can see that each layer has a

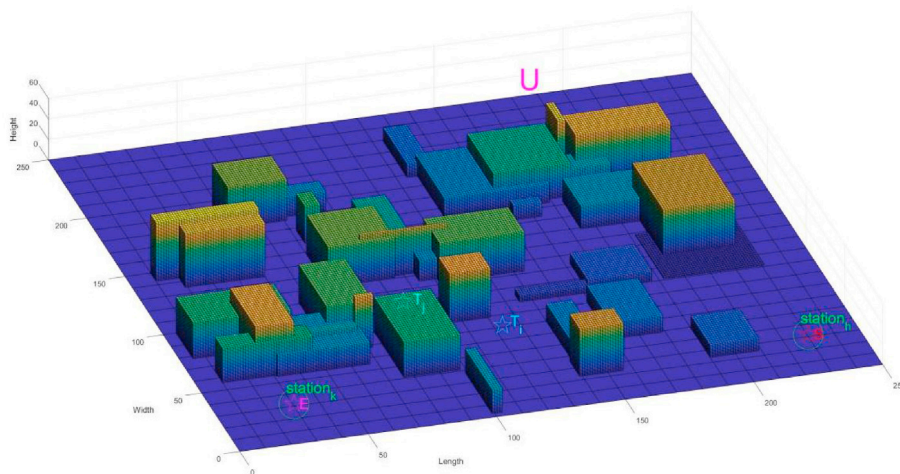


Fig. 1. 3D geometrical space with buildings of different heights.

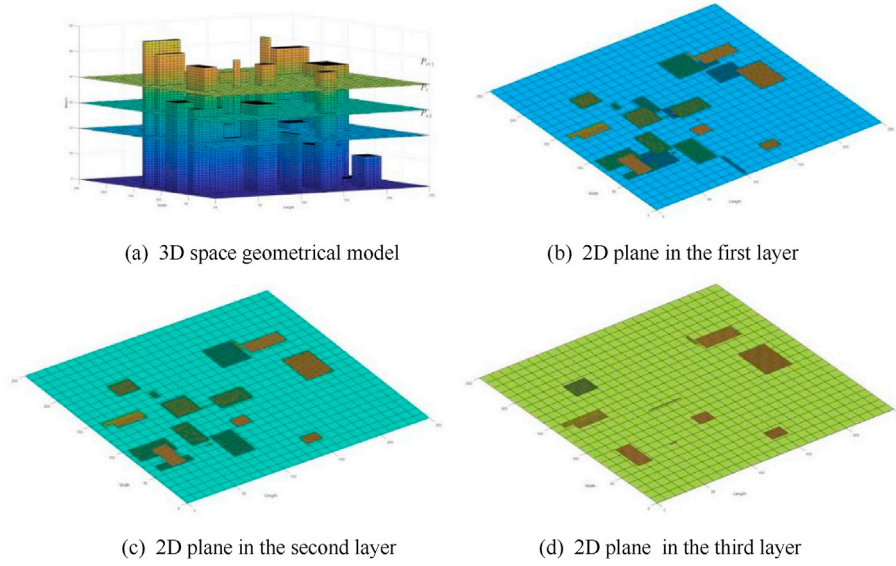


Fig. 2. The transformation of a 3D urban space into N 2D planes.

different obstacle to avoid. In other words, when the UAV flies in the 3D space in Fig. 2 (a), instead of carrying out 3D path planning, we only need to plan the path in the 2D planes in Fig. 2(b), (c) and (d) according to the height information of specific tasks. In this way, the efficiency of the path planning algorithm can be greatly improved.

3.2. UAV energy consumption model

The ultimate goal of UAV path planning is to accomplish emergency tasks in the least amount of time. This requires us to study the energy consumption of UAV. According to Ref. [38], we model a UAV's power $p(m, v, \alpha)$ as a function of its weight m , velocity v and flying direction degree α .

We randomly generated the urban environment model of 10 buildings to explain and display the energy consumption more clearly. As shown in Fig. 3, when a UAV flies vertically from point (x_i, y_i, z_i) to point (x_j, y_j, z_j) , we just need to calculate the energy consumption of its vertical flight. When it flies horizontally from point (x_j, y_j, z_j) to point (x_k, y_k, z_k) , we just need to calculate the energy consumption of its horizontal flight.

When the UAV is flying horizontally, we denote $v = v^{\parallel}$ and $\theta(v^{\parallel}, \alpha) = \theta(v^{\parallel}, 0)$. η is a constant. So the horizontal energy consumption from point (x_j, y_j, z_j) to point (x_k, y_k, z_k) is

$$p_{j \rightarrow k}(m, v^{\parallel}, 0) = \frac{mg\sqrt{(x_j - x_k)^2 + (y_j - y_k)^2}}{\theta(v^{\parallel}, 0)\eta} \quad (1)$$

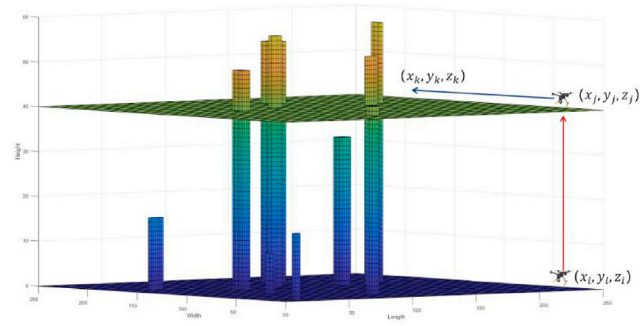


Fig. 3. Energy consumption analysis of the UAV in horizontal and vertical flight.

When the UAV is flying vertically, we denote $v = v^{\perp}$ and $\theta(v^{\perp}, \alpha) = \theta(v^{\perp}, \pi/2)$. So the vertical energy consumption from point (x_i, y_i, z_i) to point (x_j, y_j, z_j) is

$$p_{i \rightarrow j}(m, v^{\perp}, \pi/2) = \frac{mg(z_j - z_i)}{\theta(v^{\perp}, \pi/2)\eta} \quad (2)$$

Without loss of generality, when a UAV is conducting an emergency communication task or rescue operation, it will not carry out frequent altitude changes. Therefore, in the problem studied in this paper, the energy consumption required by UAV vertical flight is much smaller than horizontal flight, $p^{\perp} \ll p^{\parallel}$. In other words, the energy consumption of UAV in hierarchical path planning is dominated by the energy consumption in horizontal flight. In horizontal flight, the energy consumption of UAV is determined by the distance. Therefore, we can directly optimize the flying distance of the UAV to save energy and deliver fast service. Fig. 4 shows a 2D working environment for UAV after the 3D space transformation in section 3.1.

4. HL-PSO algorithm

4.1. Problem analysis

To distinguish the safe path and collision path in UAV global path planning and explain their relationship or among other excellent paths in each iteration process, we described them as follows in Fig. 5. It shows that the Euclidian distance between the locally optimal safety path 1 and the collision path 2 is close. Still, path 2 passes through the obstacle three

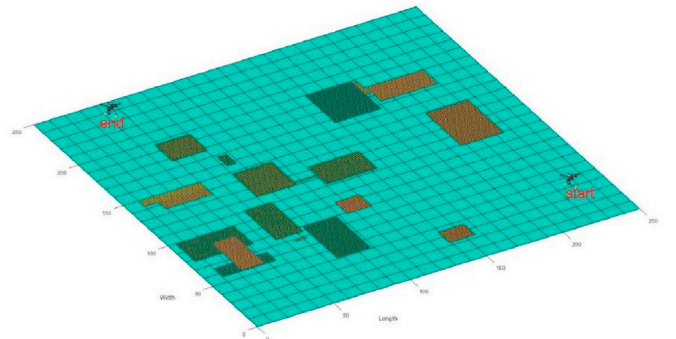


Fig. 4. The 2D working environment of UAV.

times. Path 3 is farther away from path 1 but better than path 2. Therefore, for the global path planning problem, the local optimal solutions are distributed in different regions, and the Euclidian distance between them is often far away. This paper is inspired by the above problems to improve the PSO algorithm. Therefore, for this problem, we make all kinds of particles learn from the excellent particles far away from them in different ways. In addition, we measure the diversity of algorithms to get beyond local optimality.

4.2. Basic PSO algorithm

This paper aims to study the problem of fast path planning for UAVs in urban environments. Therefore, we abstract the path of the UAV from the starting point to the ending point as a particle and obtain the optimal or sub-optimal path by iterative optimization of the PSO algorithm.

In the basic PSO algorithm, the problem's solution is abstracted into mass-less and volume-less particles and extended to the D-dimensional space, forming the particle swarm $X = (x_1, x_2, \dots, x_i, \dots, x_N)$. Where N is the population size, $i \in \{1, 2, \dots, N\}$. Each particle is described by a position $x_i = (x_i^1, x_i^2, \dots, x_i^d, \dots, x_i^D)$ and a velocity $v_i = (v_i^1, v_i^2, \dots, v_i^d, \dots, v_i^D)$, and the particle is updated over the iterations. The particle swarm gets closer to the optimal solution as the iteration continues. The update of a particle can be described as

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (x_{pbest,i} - x_i(t)) + c_2 r_2 (x_{gbest} - x_i(t)) \quad (3)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (4)$$

Where t is the number of iterations, and T represents the total number, $t \in \{1, 2, \dots, T\}$. $x_i(t+1)$ and $v_i(t+1)$ represent the position and velocity of the particle in the $(t+1)$ th iteration respectively. $x_{pbest,i}$ and x_{gbest} represent the personal historical best position of the i th particle and the global best position respectively. ω represents the inertia weight. c_1 and c_2 are the individual cognitive factor and the social cognitive factor. r_1 and r_2 are random numbers uniformly between 0 and 1. Algorithm 1 described the basic PSO in detail.

4.3. Diversity measurement

The diversity of the population is often used to evaluate the degree of convergence of the algorithm. In this section, we will design a measurement method of the diversity of the population to guide the HL-PSO algorithm in choosing different learning strategies.

$d_{g-x_i}(t)$ is assumed to be the Euclidean distance between the global optimal particle and the i th particle after the t th iteration, which can be expressed as

$$d_{g-x_i}(t) = \sqrt{\sum_{d=1}^D (x_i^d(t) - g^d(t))^2} \quad (5)$$

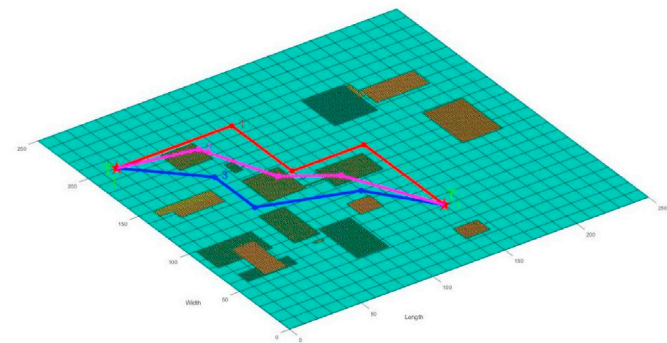


Fig. 5. Example description of Euclidean distance relationships between various paths during iterations.

It is assumed that the global optimal particle is x_N . $d_{av-g-x}(t)$ is the average Euclidean distance between the global optimal particle and all the particles in the population (except for itself) after the t th iteration, which can be expressed as

$$d_{av-g-x}(t) = \frac{1}{N-1} \sum_{i=1}^{N-1} \sqrt{\sum_{d=1}^D (x_i^d(t) - g^d(t))^2} \quad (6)$$

Assume that W is a positive integer. When $t > W$, we add the $d_{av-g-x}(t)$ of the $(t - W + 1)$ th to the t th iteration to take the average value and get formula (7).

$$d_{av-g-x-Wavg}(t) = \frac{1}{W} \sum_{i=t-W+1}^t d_{av-g-x}(i) \quad (7)$$

In this paper, through the experimental verification, when $W = 10$, the algorithm achieved better performance.

By dividing $d_{av-g-x}(t)$ by $d_{av-g-x-Wavg}(t)$, the diversity trend of the t iteration compared with the previous W iteration can be obtained. The experiment shows that the variation of diversity after the t th iteration is very large compared with the average level of diversity in the previous W iteration. To show the variation more clearly, we carry out logarithmic processing on it and get

$$f(t) = \log \left(\frac{d_{av-g-x}(t)}{d_{av-g-x-Wavg}(t)} \right) \quad (8)$$

We conducted some experiments to compare the relationship between changes in diversity levels and fitness values in Equations (3)–(8). Some standard test functions are used in the experiment. Due to the limitation of the length of the article, the experimental results of a standard test function are presented here, which is the function shown in formula (9). The minimum point of the function is 0, and the dimension is $D = 30$.

$$f_1 = \sum_{i=0}^D x_i^2 \quad (9)$$

Fig. 6 shows the relationship between the population diversity trend and the function's fitness value in 200 iterations. Fig. 7 shows the relationship between the population diversity trend and the function's fitness value in the 1000 iteration. As can be seen from Figs. 6 and 7, when $f(t)$ shows an upward trend, the function converges quickly; when the diversity is not an upward trend, the function converges slowly, or the fitness value of the function is almost unchanged.

To describe the trend of diversity concretely, we use the least square method to perform the linear fitting for the W values $f(t-W+1), f(t-W+2), \dots, f(t)$ of each W generation. Thus, in each iteration, we can obtain the slope of a fitting line $k(t)$. When $k(t) > 0$, it indicates that the diversity is increasing, and when $k(t) < 0$, it means the variety is not growing.

4.4. Learning samples

In Section 4.3, we have obtained the Euclidean distance d_{g-x_i} between each particle and the global optimal particle. d_{g-x_i} is normalized as

$$p_{g-x_i} = \frac{d_{g-x_i}}{d_{g-x_1} + d_{g-x_2} + \dots + d_{g-x_{N-1}}} \quad (10)$$

Take the reciprocal of the distance between each particle and the global optimal particle $1/d_{g-x_i}$, and then $1/d_{g-x_i}$ is normalized as

$$p'_{g-x_i} = \frac{1/d_{g-x_i}}{1/d_{g-x_1} + 1/d_{g-x_2} + \dots + 1/d_{g-x_{N-1}}} \quad (11)$$

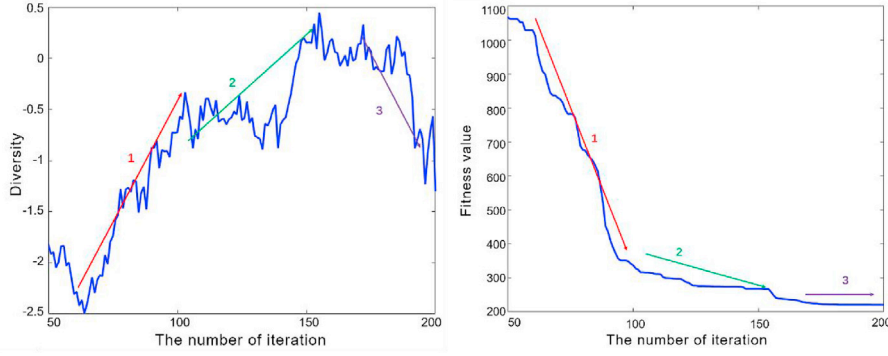


Fig. 6. (a) The trend of population diversity in 200 iterations; (b) The trend of fitness value in 200 iterations.

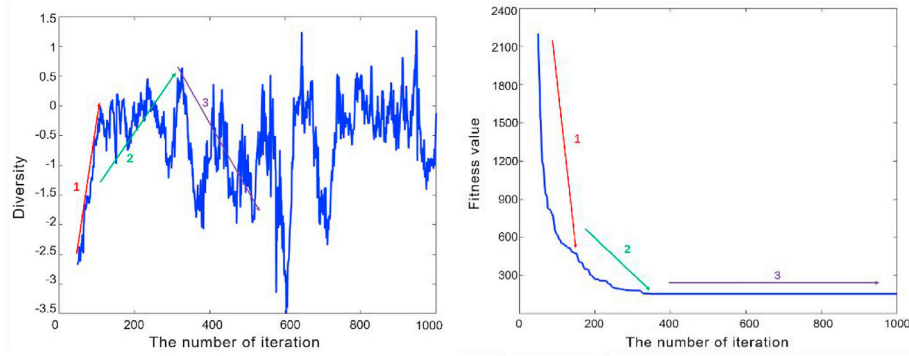


Fig. 7. (a) The trend of population diversity in 1000 iterations; (b) The trend of fitness value in 1000 iterations.

The fitness value of each particle $fit(i)$ is normalized as

$$q_{x_i} = \frac{fit(i)}{fit(1) + fit(2) + \dots + fit(N)} \quad (12)$$

If k_{g-x_i} represents a kind of comprehensive ability of the particle, the closer it is to the global optimal particle, the greater the fitness value and the comprehensive ability are. Then k_{g-x_i} can be expressed as

$$k_{g-x_i} = \alpha_1 p'_{g-x_i} + \alpha_2 q_{x_i} \quad (13)$$

k'_{g-x_i} is used to represent another kind of comprehensive ability of the particle. The farther it is away from the global optimal particle, the greater the fitness value and the comprehensive ability are. Then k'_{g-x_i} can be expressed as

$$k'_{g-x_i} = \alpha_1 p_{g-x_i} + \alpha_2 q_{x_i} \quad (14)$$

In selecting learning samples, we sort all particles based on k_{g-x_i} and k'_{g-x_i} . Then we choose a certain number of learning samples according to the actual situation of the algorithm. If it is in the exploration stage, the sample with larger comprehensive energy k_{g-x_i} is selected; if it is in the development stage, the particle with larger k_{g-x_i} and k'_{g-x_i} is selected with a certain probability to maintain the global and local search balance.

4.5. Algorithm model

Based on the above, we described in detail the proposed HL-PSO model in this section. The main idea of the algorithm is hierarchical interactive learning. That is, different levels of particles have different learning strategies and learning samples.

According to the k_{g-x_i} and k'_{g-x_i} , we sort for all particles, and take the biggest M particles as learning samples: $y_1, y_2, \dots, y_M$ and $y'_1, y'_2, \dots, y'_M$ respectively.

For the global optimal particle, we set two learning strategies. When $k(t) > 0$ (the population diversity is on the rise), the algorithm has a strong exploration ability. We take the particle with a bigger k_{g-x_i} as the learning sample of the global optimal particle. That is to say, the global optimal particle can learn from the particle with a large fitness value closer to it, to improve its local search ability. When $k(t) \leq 0$ (the population diversity shows a non-increasing trend), the particles of the larger k_{g-x_i} or k'_{g-x_i} are selected as the learning samples of the global optimal particles with a certain probability, to balance the global and local search ability.

When $k(t) > 0$, the velocity update formula of the global optimal particle can be expressed as

$$v_{gbest}(t+1) = wv_{gbest}(t) + c_1 r_1 k_{g-y_1} (y_1 - x_{gbest}) + \dots + c_M r_M k_{g-y_M} (y_M - x_{gbest}) \quad (15)$$

When $k(t) \leq 0$, the velocity update formula of the global optimal particle can be expressed as

$$\begin{aligned} v_{gbest}(t+1) = & wv_{gbest}(t) \\ & + [q + r] \left[c_1 r_1 k_{g-y_1} (y_1 - x_{gbest}) + \dots \right. \\ & \left. + c_M r_M k_{g-y_M} (y_M - x_{gbest}) \right] \\ & + (1 - [q + r]) \left[c_1 r_1 k'_{g-y_1} (y'_1 - x_{gbest}) + \dots \right. \\ & \left. + c_M r_M k'_{g-y_M} (y'_M - x_{gbest}) \right] \end{aligned} \quad (16)$$

For other particles, we also design two learning strategies. When $k(t) > 0$, the learning samples of this particle are the global optimal particle position and the individual historical optimal position. When

$k(t) \leq 0$, the learning samples of this particle are the global optimal particle, the individual historical optimal position, and $M-2$ particles with bigger k'_{g-x_i} . The learning strategies for the other particles are described in detail below.

When $k(t) > 0$, the velocity update formula of the other optimal particles can be expressed as

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (x_i - x_{pbest,i}) + c_2 r_2 (x_i - x_{gbest}) \quad (17)$$

When $k(t) \leq 0$, the velocity update formula of the other optimal particles can be expressed as

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (x_i - x_{pbest,i}) + c_2 r_2 (x_i - x_{gbest}) + (1 - [q + r]) [c_3 r_3 k'_{g-y_1} (y'_1 - x_{gbest}) + \dots + c_M r_M k'_{g-y_{M-2}} (y'_{M-2} - x_{gbest})] \quad (18)$$

According to the experimental verification, we set other parameters in this paper as $w = 0.6$, $r_1, r_2, \dots, r_M \in [0, 1]$. In Equations (15), (16) and (18), $c_1 + c_2 + \dots + c_M = 4, c_i = 4/M$. In Equation (17), $c_1 = c_2 = 2$. In Equations (16) and (18), $q = 0.7$.

The PSO algorithm is a stochastic algorithm where the initial positions and initial velocities of the particles are random values. Of course, the range of its random values is different for different problems. For example, in this paper we experimentally set the range of values of the velocity to be one-tenth of the range of values of the position.

The pseudo-code of the improved algorithm is shown in Algorithm 1.

Algorithm 1 : HL-PSO algorithm

- 1: Initialize a population of particles X .
 - 2: Calculate the fitness of all particles $f(X)$.
 - 3: Set $x_{pbest,i}$ to be the personal historical best position of the i^{th} particle.
 - 4: Set x_{gbest} to be the global best position.
 - 5: **While** not termination condition do
 - 6: **For** $i = 1:N$, do
 - 7: Sample learning samples according to Equation (13) and (14).
 - 8: Analyze the diversity through Equation (8) and compute $k(t)$.
 - 9: **If** $k(t) > 0$ or $t < W$
 - 10: Update v_i according to Equation (15) and (17).
 - 11: Update x_i according to formula (4).
 - 12: **End if**
 - 13: **If** $k(t) \leq 0$
 - 14: Update v_i according to formula (16) and (18).
 - 15: Update x_i according to formula (4).
 - 16: **End if**
 - 17: **End for**
 - 18: **End while**
-

4.6. Experimental verification and result analysis

To verify the performance of the HL-PSO algorithm proposed in this paper, we used a complex obstacle environment to conduct comparative experiments on the basic PSO algorithm, CLPSO algorithm [39], SALMPSO algorithm [40], and HL-PSO algorithm. We randomly generated 100 pairs of starting and ending points in the experiment, respectively. We run the algorithm for 500 generations at a time and average the results by running the algorithm 100 times. The relationship between the average length of 100 paths planned by the three algorithms and the number of iterations was obtained, as shown in Fig. 8.

From Fig. 8, we can see that the HL-PSO algorithm proposed in this chapter has a faster convergence speed and higher accuracy. To further compare the above four algorithms, we use Table 1 to show the average length of successful paths planned by the four algorithms and the average planning time. As seen in Table 1, although the basic PSO algorithm takes the least time, it has the longest average length and the lowest success rate of the successful path planned by it. The successful path here refers to the collision-free safe path, and the success rate refers to the proportion of the safe collision-free path. While the HL-PSO algorithm takes the longest time, it has the shortest average length of the successful path and the highest success rate. Besides, we can see that the average planning time of CLPSO and SALMPSO is shorter than that of HL-PSO, but their performance in terms of success rate and the average length of the successful path is inferior to that of the HL-PSO algorithm. Because the algorithm is designed for the global path planning of the UAV, the global path planning attaches more importance to accuracy than the planning, so the performance of the HL-PSO algorithm is better than the other three algorithms. The endurance of each UAV's onboard energy storage is fundamentally limited by its weight and aircraft size. HL-PSO algorithm in this paper aims to find a collision-free shortest path. The algorithm convergence speed and accuracy of the HL-PSO algorithm are higher than that of the PSO algorithm. Thus, by using HL-PSO, more energy from UAV can be saved, and the time of endurance will be extended.

5. TSEB-DDPG algorithm

Given the UAV fast path planning problem in the urban environment studied in this paper, UAV is regarded as an agent, and the empirical path obtained by iterative optimization of the HL-PSO algorithm is used as the guidance of the agent to advance. The improved DDPG algorithm is used to train the fast path planning model. The model is described in detail below.

5.1. Basic DDPG algorithm

The algorithm flow diagram of the DDPG algorithm is shown in Fig. 9.

As shown in Fig. 9, Actor gives a deterministic action strategy $a_t = \mu(s_t; \theta^\mu) + \mathcal{N}$, after which the agent performs the action a_t , gets the reward r_t , and enters the next state s_{t+1} . Actor stores the transition information (s_t, a_t, r_t, s_{t+1}) of the agent into the experience buffer D . $\mathcal{N}'$

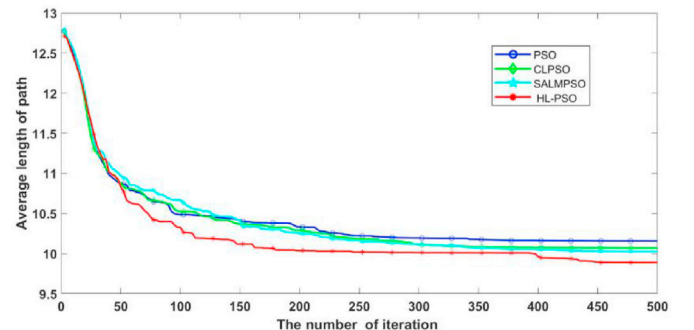


Fig. 8. Comparison of the experimental results of the four algorithms.

Table 1

Performance comparison of four algorithms.

Algorithms	Average length of the successful path (m)	Average planning time (s)	Success rate (%)
PSO	9.81	82	78.6
CLPSO	9.73	96	81.3
SALMPSO	9.71	102	82.1
HL-PSO	9.61	113.6	86.1

represents the Ornstein-Uhlenbeck noise. Ornstein-Uhlenbeck noise is a mean-reversion process, meaning states are pulled back when they deviate from the mean.

The policy network μ and value network Q uniformly sample N transition information in D to train their parameters. The loss function in the Q network is defined as

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i; \theta^Q))^2 \quad (19)$$

Where y_i is obtained by target policy network μ' and target value network Q' . If the training is terminated in the $(i+1)^{th}$ episode, $y_i = r_i$, else y_i can be expressed as

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}; \theta^{\mu'}); \theta^{Q'}) \quad (20)$$

Where θ^μ and θ^Q are the parameters of the Actor and Critic network, respectively, and $\theta^{\mu'}$ and $\theta^{Q'}$ are the parameters of the target Actor and Critic network, respectively.

After that, the Critic network is updated by minimizing the loss function L in Equation (19). And then the sampling strategy gradient is used to update the Actor network through the following formula

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a; \theta^Q) \big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s; \theta^\mu) \big|_{s=s_i} \quad (21)$$

Finally, the two target networks are updated according to

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \quad (22)$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \quad (23)$$

The pseudo-code of the DDPG algorithm is described in Algorithm 2.

The DDPG algorithm mainly includes several key points.

- The DDPG algorithm is a deep reinforcement learning algorithm based on the Actor-Critic framework to solve the continuous control problem.
- The DDPG algorithm uses the experience buffer to save the historical transition data, and the data in the experience buffer is sampled and trained by the deep neural network to obtain the parameters of the Actor and Critic network.
- The DDPG algorithm uses the addition of noise to expand the exploration space of action. The experience buffer is the key to combining deep learning and reinforcement learning.

The experience buffer stores the past transition, accessed in the form of advanced first. The new transition is kept in the experience buffer, the information on the remote transition is abandoned, and the capacity of the experienced buffer is unchanged. During training, data in the experience buffer is randomly sampled. The buffer memory track in a period can ensure the stable distribution of training data. Forgetting remote trajectories ensures that the distribution of sampled data is most similar to the current situation.

DDPG uses uniform sampling to sample the experience buffer in small batches. This kind of uniform sampling will result in poor sample quality and diversity, leading to low learning efficiency and a slow algorithm convergence rate. In addition, in the process of learning, because the action space of DDPG is added with regular noise, the DDPG algorithm is not easy to jump off when the agent falls into a dilemma. We proposed a three-step experience buffer DDPG algorithm based on the above reasons. To improve the training speed and accuracy of the DDPG algorithm, we designed three experience buffers for the DDPG algorithm: the excellent transition buffer, the collision transition buffer, and the fast transition buffer. In addition, we dynamically change the sampling ratio of each experience buffer according to the training stage. The improved algorithm is described in detail as follows.

5.2. Three experience buffers

The basic DDPG algorithm uses an experience buffer to store past experiences. Each experience is represented by a quad (s_t, a_t, r_t, s_{t+1}) . The meaning expressed by this quadruple is that the agent performs an action

Algorithm 2: DDPG algorithm

- 1: Randomly initialize Critic network $Q(s, a; \theta^Q)$ and Actor network $\mu(s; \theta^\mu)$ with θ^Q and θ^μ .
 - 2: Initialize target network Q' and μ' with $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$.
 - 3: Initialize Experience buffer D .
 - 4: **For** episode = 1 to M , **do**
 - 5: Initialize noise $\mathcal{N}$ for action exploration.
 - 6: Receive initial observation state s_1 .
 - 7: **For** $t = 1$ to T , **do**
 - 8: Select action a_t according to the current policy and exploration noise.
 - 9: Execute action a_t and observe reward r_t and observe new state s_{t+1} .
 - 10: Store transition (s_t, a_t, r_t, s_{t+1}) to experience buffer D .
 - 11: Sample a random minibatch of N transition (s_i, a_i, r_i, s_{i+1}) from D .
 - 12: y_i is obtained from Equation (20).
 - 13: Critic network is updated by minimizing the loss function L of Equation (19).
 - 14: The sampling policy gradient of Equation (21) is used to update Actor network.
 - 15: Two target networks are updated by Equations (22) and (23).
 - 16: **End for**
 - 17: **End for**
-

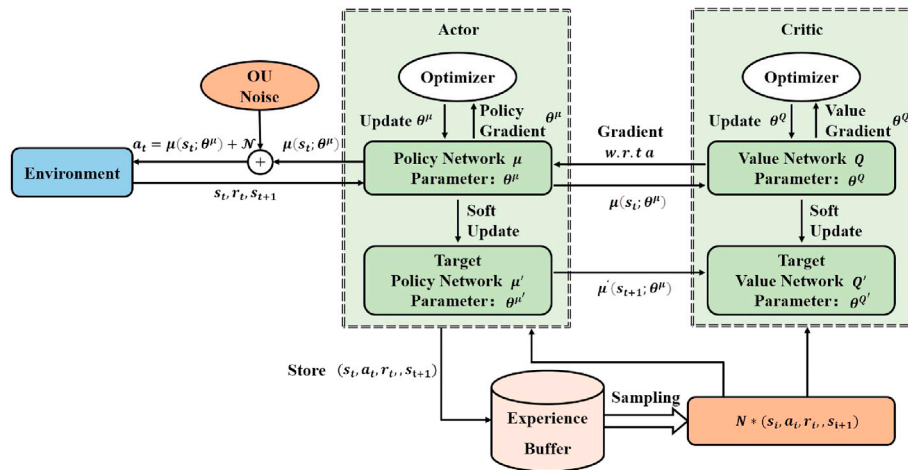


Fig. 9. The flow chart of DDPG algorithm.

a_t in state s_t , gets reward r_t and moves to state s_{t+1} .

Three experiential buffers are set up in the improved DDPG algorithm: the excellent transition buffer, the collision transition buffer, and the fast transition buffer. After the excellent global path information obtained by the HL-PSO algorithm is converted to the quadruple information matching the experience buffer, it is stored in the excellent transition buffer. The collision transition buffer saves the collision information obtained from real training, which is recognized in the algorithm training process and stored in the collision experience buffer in time. The information stored in the fast transition buffer is the fast path information, which is automatically stored in training, and its method is the same as the basic DDPG algorithm.

There are three reasons for designing three different types of experience buffers.

- To improve the learning efficiency of agents, effective prior knowledge must be added to the experience buffer. Without valid prior knowledge, the agent can only learn by trial and error. If we set fewer learning times for the agent during the training, it is possible that the agent cannot complete effective learning at the end of the training, which will directly lead to the failure of the training task. Even if we set too many learning times for the agent, it will cause random trial-and-error learning of the agent in the early stage of training, and it will not be able to conduct efficient learning. This will lead to problems such as low learning efficiency, non-convergence, and poor accuracy. The above issues will become larger with the increase of the action space of the agent. For continuous action space, making the algorithm converge without prior knowledge as the guide is difficult.
- To improve the knowledge reserve of the agent, it is necessary to ensure the importance and diversity of the data sampled from the experience buffer each time. In many operating environments of agents, the changes in the environment may be very small, which results in a very strong correlation of the track information (s_t, a_t, r_t, s_{t+1}) in the experience buffer that is accessed and sampled sequentially. It is impossible to distinguish the importance of each piece of information, and important information will be lost under the FIFO access mechanism. Therefore, to ensure that every kind of important information is sampled, we specially prepared an excellent track experience buffer for the excellent path information planned by the HL-PSO algorithm. The agent not only needs to learn about good experiences but also about bad experiences. Therefore, we have prepared a collision experience buffer for the agent. The agent is constantly exploring, trying, and making mistakes. The collision information will be stored in the collision experience buffer once resolved.

- To improve the exploration ability of the agent, only excellent and collision knowledge is not enough. Because imagination can sometimes produce new sparks of knowledge, if the agent is guided only by excellent knowledge and collision knowledge, it would be controlled by these two experts and lose its vitality. Therefore, in addition to setting up the excellent transition buffer and collision transition buffer to provide expert guidance for the agent, we also set up a fast transition buffer to learn the track it has explored before.

5.3. Three-step sampling mechanism

To make the three experience buffers cooperate, we learned from the process of children learning to walk to adjust the sampling proportion of the three buffers adaptively. Children are held and led by their elders in the early stage of learning to walk. When they encounter obstacles, they will be reminded. However, children will sometimes lead their elders to walk around according to their own will to explore the world. This stage is called the *toddler stage*. When children can waddle, the elders will help them occasionally and only remind them when they are about to fall or tell them to be careful the next time they fall. This stage is called the *waddle stage*. When children have learned to walk completely, the elders no longer need to instruct them, and they can walk by themselves. This stage we call the *victory stage*. Drawing on this idea, we set the sampling ratio of the three buffers in a step-by-step manner according to the toddler stage, waddle stage and victory stage. Since the algorithm uses stepped sampling in three stages of algorithm training, the sampling mechanism is called a three-step sampling mechanism.

To describe the improvement strategy more clearly, let's first make some assumptions.

- Assume that the agent needs to complete training in Episode L .
- Before each training, the agent analyzes the average reward obtained from the previous F training and uses the least square method to fit a straight line for the average reward of the F training (in this paper, $F = 5$). The slope of the line is set as p .
- According to the actual problem and the experimental situation, a threshold value e_1 is set for the slope p , and e_1 is set as 1 in this paper.
- The average reward for excellent trajectories in the excellent transition buffer is $\overline{R}_{best}$.

The learning stage of the agent is divided according to the following method. When $p < e_1$ and the reward $r \leq \overline{R_{best}}$, the agent is in the toddler stage. When $p \geq e_1$, the agent is in the waddle stage. When the reward $r > v \cdot \overline{R_{best}}$, the agent is in the victory stage. In this paper, $v = 0.5$. The sample ratio for each experience buffer is designed according to these three stages.

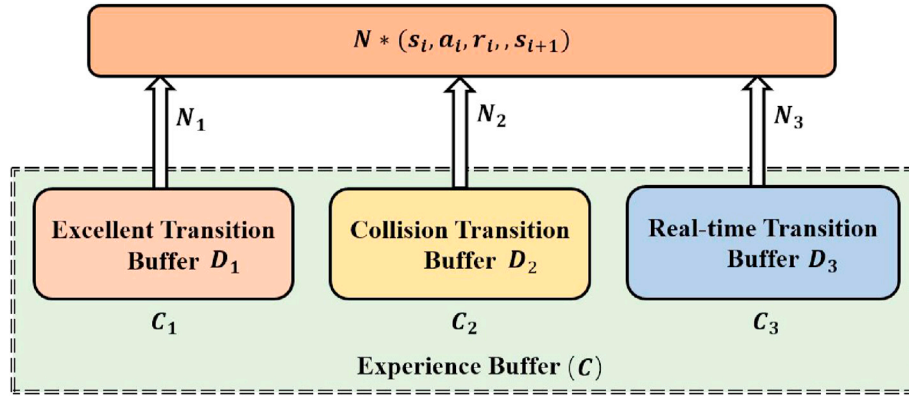


Fig. 10. Principle block diagram of three-step sampling mechanism.

Assume that the total capacity of the experience buffer is C , and the capacity of the excellent transition buffer, the collision transition buffer, and the fast transition buffer are C_1 , C_2 , and C_3 , respectively. It must satisfy $C = C_1 + C_2 + C_3$. If the mini-batch of samples from the experience buffer is assumed to be N , then the number of samples from the three buffers are N_1 , N_2 , and N_3 , respectively, and $N = N_1 + N_2 + N_3$ must be satisfied. In this paper, $C = 1000$, $C_1 = 100$, $C_2 = 100$, $C_3 = 800$, $N = 100$, and the values of N_1 , N_2 and N_3 are functions that change with the increase of training times. Specific settings of N_1 , N_2 and N_3 can be expressed as

$$N_1 = \begin{cases} 0.5(N - i), & p < e_1 \\ 0, & p \geq e_1 \end{cases} \quad (24)$$

$$N_2 = \begin{cases} 0.4(N - i), & r \leq v \cdot \bar{R}_{best} \\ 0, & r > v \cdot \bar{R}_{best} \end{cases} \quad (25)$$

$$N_3 = N - N_1 - N_2 \quad (26)$$

In other words, the excellent transition buffer only plays a guiding role in the toddler stage. The collision transition buffer plays a guiding role in both the toddler stage and the waddle stage, while the fast transition buffer plays a guiding role in all stages. In the victory stage, the fast transition buffer independently guides the agent's learning.

The principle block diagram of the three-step sampling mechanism is shown in Fig. 10. The specific process of the three-step sampling mechanism is shown in Fig. 11.

5.4. Reward mechanism

In this paper, it is assumed that the speed of the UAV is constant, and the direction of motion of the UAV can be controlled by the turning Angle φ , $\varphi \in [-\pi, \pi]$. The motion space of the UAV is continuous, and its state is its 2D coordinate. Since the motion space is continuous, the state space of the UAV is also continuous. Because the state space is continuous, it is difficult to find the exact endpoint coordinates. In practical applications, if the UAV is near the endpoint, we consider the UAV to have reached the endpoint. Therefore, when the distance between the UAV and the endpoint is less than the threshold value, we consider the UAV to have reached the endpoint and give the UAV a large positive reward.

We combined the HL-PSO and the DDPG algorithm to meet the fast and accuracy requirements of UAV path planning.

The reward mechanism is very important for the RL algorithm because only under the positive guidance of reward strategy can an agent successfully complete training. A reasonable reward strategy has a practical guiding significance for the UAV's next action. In the continuous action space of the UAV, if the UAV is rewarded only when it finds the endpoint, it will lead to too few positive samples and low learning efficiency. If the UAV gives a reward for every collision-free step, the process

reward will accumulate and overwhelm the target reward. This would cause the UAV to stray from its target, score points by avoiding obstacles, and fall into aimless, never-ending wandering. We design the following reward mechanism to solve this problem based on the step-forward relationship.

Assume that the starting point of a UAV is *Start*, the target point is *End*, the coordinate of the current position is S_t , the coordinate of the previous position is S_{t-1} , and the total interval of static obstacles in the UAV's walking process is Z . Then, the distance between S_t and *End* is $dis(S_t, End)$, and the distance between S_{t-1} and *End* is $dis(S_{t-1}, End)$. The system rewards the UAV in the following way:

- When $S_{t-1} \in Z$ and $S_t \in Z$, i.e., the UAV collides with an obstacle in the first step and fails to correct the obstacle in the last step, the UAV is given a negative reward: $Reward = -d_1$.
- When $S_{t-1} \in Z$ and $S_t \notin Z$, i.e., the UAV collides with an obstacle in the first step and avoids the obstacle effectively in the last step, the UAV is given a positive reward: $Reward = d_1$.
- When $S_{t-1} \notin Z$ and $S_t \in Z$, i.e., the UAV does not collide with the obstacle in the previous step but bumps into the obstacle in the next step, the UAV is given a negative reward because it has not learned enough obstacle avoidance techniques: $Reward = -d_1$.
- When $S_{t-1} \notin Z$, $S_t \notin Z$, and $dis(S_{t-1}, End) < dis(S_t, End)$, i.e., the UAV does not collide with the obstacle in either step, but it is far away from

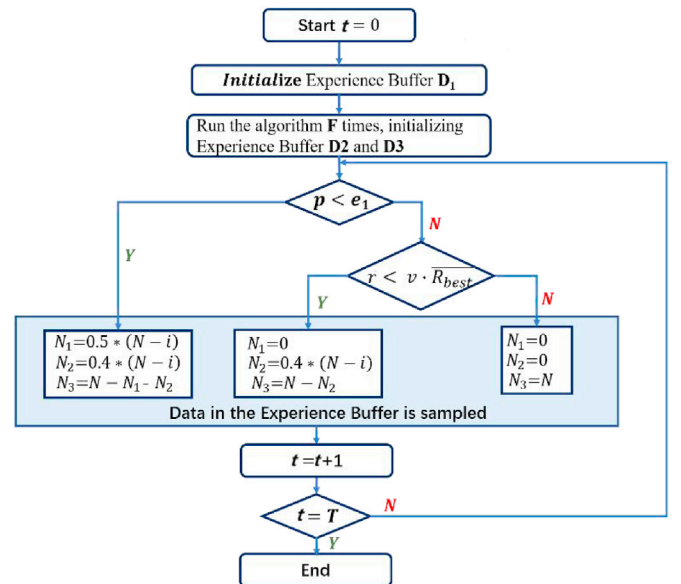


Fig. 11. Sampling flow chart of three-step mechanism.

the target, therefore, the UAV is given a negative reward: $Reward = -d_1$.

- When $S_{t-1} \notin Z$, $S_t \notin Z$, and $dis(S_{t-1}, End) > dis(S_t, End)$, i.e., the UAV does not collide with an obstacle one step ahead or the next, it gets closer and closer to the target, so the UAV is given a positive reward: $Reward = d_1$.
- To push the UAV towards End , when the UAV is approaching its destination, it satisfies $dis(S_t, End) < k_1$. We give it a progressive reward: $Reward = d_2$.
- If the UAV is very close to End , it satisfies $dis(S_t, End) < k_2$. It will be given the maximum positive reward: $Reward = d_3$.

The above parameters d_1, d_2, d_3, k_1 and k_2 can be debugged according to actual application scenarios. They should meet the following requirements: $d_1 < d_2 \ll d_3, k_1 < k_2$.

After the above improvements, we can get the pseudo-code of the TSEB-DDPG algorithm as described in Algorithm 3.

Algorithm 3: TSEB-DDPG algorithm

```

1: Randomly initialize Critic network  $Q(s, a; \theta^Q)$  and Actor network  $\mu(s; \theta^\mu)$  with  $\theta^Q$  and  $\theta^\mu$ .
2: Initialize target network  $Q'$  and  $\mu'$  with  $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$ .
3: Initialize Experience buffer  $D_1$ .
4: For episode = 1 to  $M$ , do
5:   Initialize noise  $\mathcal{N}$  for action exploration.
6:   Receive initial observation state  $s_1$ .
7:   For  $t = 1$  to  $F$ , do
8:     Select action  $a_t$  according to the current policy and exploration noise.
9:     Execute action  $a_t$  and observe reward  $r_t$  and observe new state  $s_{t+1}$ .
10:    Collision information is identified and the collision transition information is stored
        in the experience buffer  $D_2$ .
11:    The fast transition information is stored in the experience buffer  $D_3$ .
12:   End for
13:   For  $t = F+1$  to  $T$ , do
14:     Select action  $a_t$  according to the current policy and exploration noise.
15:     Execute action  $a_t$  and observe reward  $r_t$  and observe new state  $s_{t+1}$ .
16:     Collision information is identified and the collision transition information is stored
        in the experience buffer  $D_2$ .
17:     The fast transition information is stored in the experience buffer  $D_3$ .
18:     According to the sampling mechanism of
        Equations (24) to (26), Sample a random minibatch of  $N$  transition( $s_i, a_i, r_i, s_{i+1}$ ) from
        three experience buffers.
19:      $y_i$  is obtained from Equation (20).
20:     Critic network is updated by minimizing the loss function  $L$  of Equation (19).
21:     The sampling policy gradient of Equation (21) is used to update Actor network.
22:     Two target networks are updated by Equation (22) and (23).
23:   End for
24: End for

```

In theory, the improved DDPG algorithm in this paper will improve the training speed and accuracy of the algorithm and increase the success rate of the actual path planning task. We summarize the reasons as follows.

- All kinds of prior knowledge are classified and stored to ensure that all kinds of knowledge are collected in each sampling, and the balance of sampling data is guaranteed.
- A large amount of trial-and-error time will be reduced because of prior knowledge.
- The stepwise withdrawal of prior knowledge will enable the agent to maintain its vitality and will not fall into local optimality because of prior knowledge.

6. Experiment and analysis

We set the number of training sessions to 1000, and the training terminates when one of the following two conditions is met.

- (1) The UAV reaches the end;
- (2) The UAV took 2000 steps.

The learning rates of the Actor and Critic networks are 0.0001 and 0.001, respectively. The cumulative reward discount factor is 0.9. The target Actor network and the target Critic network are updated every 100 steps with an update parameter $\tau = 0.001$.

6.1. Comparison experiment of TSEB-DDPG algorithm and other DDPG algorithms

In order to verify the performance of the proposed algorithm, we

applied the DDPG algorithm, IDDPG algorithm [41], PDP-DDPG algorithm [42] and TSEB-DDPG algorithm to the simulation experiment of UAV fast path planning.

Fig. 12 shows the comparison between the DDPG algorithm and the TSEB-DDPG algorithm proposed in this paper.

Fig. 12 shows that the average returns of the two algorithms present an irregular state of turbulence, and even the average returns become smaller in the toddler stage. This is the result of free exploration in the case of inexperienced agents. However, the TSEB-DDPG algorithm proposed in this paper jumps from the toddler stage to the waddle stage faster than the DDPG algorithm. In the waddle stage, its convergence rate is faster than that of the DDPG algorithm, and it enters the victory stage faster. After more than 200 episodes, the TSEB-DDPG algorithm converges. However, the DDPG algorithm converges after 450 episodes, and its average return is lower than the TSEB-DDPG algorithm. Therefore, the TSEB-DDPG algorithm proposed in this paper is superior to the DDPG

algorithm in terms of convergence speed and accuracy.

Fig. 13 shows the experimental comparison between the IDDPG algorithm and the TSEB-DDPG algorithm. IDDPG algorithm is improved on the DDPG algorithm and adopts a variable capacity experience buffer. As we can see from Fig. 13, compared with the DDPG algorithm, the IDDPG algorithm has improved performance and faster convergence speed. But compared with the TSEB-DDPG algorithm, its convergence speed is slower, and its accuracy is lower.

Fig. 14 shows the experimental comparison between the PDP-DDPG algorithm and the TSEB-DDPG algorithm. The PDP-DDPG algorithm selects high-priority samples to train in the network, deleting similar samples in the buffer and reserving a small number of rare samples simultaneously, which greatly improves the quality and diversity of samples. Therefore, it can be seen from Fig. 14 that the convergence speed of the PDP-DDPG algorithm is also greatly improved compared with the DDPG algorithm. But the convergence speed and accuracy of the PDP-DDPG algorithm are lower than that of the TSEB-DDPG algorithm.

In summary, the TSEB-DDPG algorithm performs better than DDPG, IDDPG and PDP-DDPG algorithms. The reason is that the TSEB-DDPG algorithm breaks down the bottleneck of the DDPG algorithm one by one in each learning stage of the training process to guide the algorithm to converge quickly. Because of the guidance of excellent experience and the timely exit of excellent experience, the algorithm's accuracy is improved, and the vitality of the algorithm is ensured.

6.2. Simulation experiment and analysis of fast path planning for UAV

Compared to the HL-PSO algorithm, the convergence speed of the DDPG algorithm is much higher. Once the DDPG algorithm is trained, it can search for optimization quickly. In the era of artificial intelligence, we hope that the UAV can promptly respond to the requirements of the environment and humans and complete the task with the optimal or sub-optimal path.

Hardware configuration for simulation experiments: GPU: Tesla K80 with 11 GB video memory; CPU: Intel(R) Xeon (R) CPU E5-2609 v4 @ 1.70 GHz; experimental framework: Pytorch. We used the HL-PSO algorithm, DDPG algorithm, and TSEB-DDPG algorithm respectively to carry out fast path planning for the UAV in the 2D plane shown in Fig. 4. Each algorithm planned 100 paths fast with a random starting point and endpoint. A comparison of the average length of these paths, the average time of path planning, and the success rate are shown in Table 2. From Table 2, we can see that the path planned by the HL-PSO algorithm has the highest accuracy, but the algorithm takes the longest time. DDPG algorithm takes the shortest time to plan the path, but its training time is longer, and its accuracy is the lowest. Compared with the DDPG algorithm, the TSEB-DDPG algorithm has a shorter training time and higher accuracy. Compared with the HL-PSO algorithm, the TSEB-DDPG

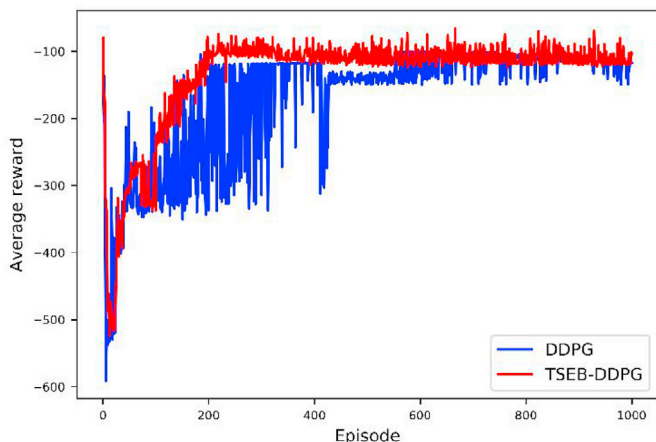


Fig. 12. Performance comparison between DDPG and TSEB-DDPG algorithms.

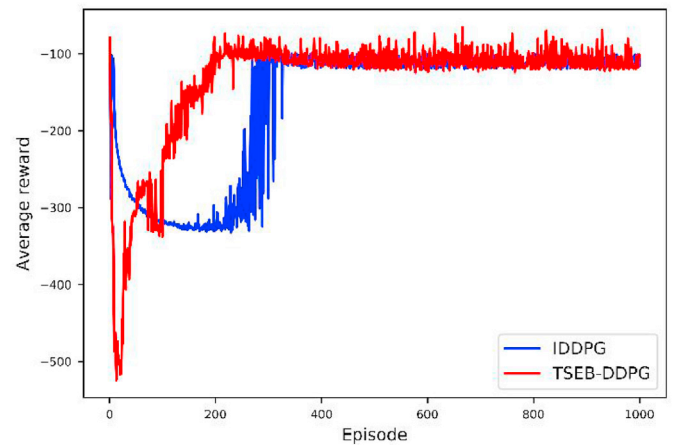


Fig. 13. Performance comparison between IDDPG and TSEB-DDPG algorithms.

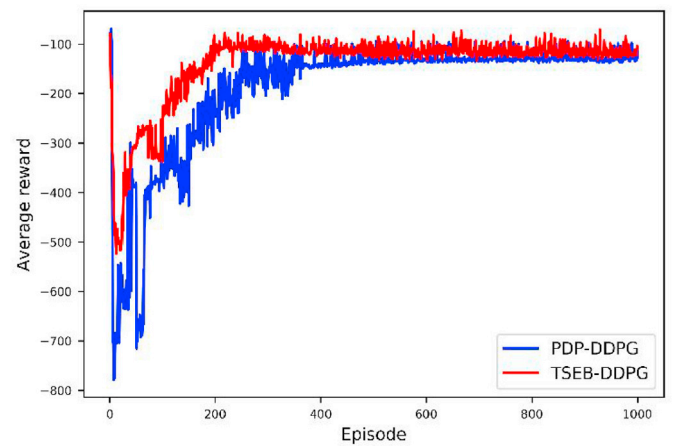


Fig. 14. Performance comparison between PDP-DDPG and TSEB-DDPG algorithms.

algorithm can meet UAV's fast planning requirements. In addition, with the complex UAV movement environment, the HL-PSO algorithm's running time will increase exponentially. At the same time, the offline training and online planning methods adopted by the TSEB-DDPG algorithm can also meet the fast requirements in the case of a complex environment. In addition, from the last column of Table 2, we can see that the TSEB-DDPG algorithm has the highest success rate in path planning. Of course, if the HL-PSO algorithm is given more time, the path planned by the HL-PSO algorithm will surely have a higher success rate. However, since the problem targeted in this paper is fast path planning, we set fewer iterations for the HL-PSO algorithm to optimize the path. Therefore, the TSEB-DDPG algorithm has a higher cost performance than the HL-PSO algorithm and the DDPG algorithm. As we can see from

Table 2

Path planning performance comparison of HL-PSO algorithm, DDPG algorithm and TSEB-DDPG algorithm.

Algorithm	Parameters	Average training time	Average planning time (s)	Average length of the successful path (m)	success rate (%)
HL-PSO	500 episodes	115.6s	115.6	36.91	86.4
DDPG	1000 episodes	20.47h	9.98	45.65	81.2
TSEB-DDPG	1000 episodes	23.73h	9.98	38.76	87.6

Table 2, TSEB-DDPG outperforms other competitors in terms of path planning time, length of planned paths and success rate.

7. Conclusion

In this paper, a three-step experience buffer DDPG algorithm is proposed to solve the fast UAV path planning problem. The proposed TSEB-DDPG algorithm guides agents to carry out effective training through three kinds of prior knowledge. An excellent transition buffer provides an excellent experience, a collision transition buffer provides a vigilant experience, and a fast transition buffer provides an immediate experience. Three experience buffers adopt a stepped sampling mechanism, enabling the agents to absorb prior knowledge while maintaining vitality. Based on the extensive experiments, the results show that the TSEB-DDPG algorithm can achieve the fastest convergence and the highest accuracy. Compared to the HL-PSO algorithm, DDPG algorithm and TSEB-DDPG algorithm for UAV fast path planning, it is found that the TSEB-DDPG algorithm is the most cost-effective. Besides, this algorithm can be extended to any single intelligent control problem with static obstacles. However, the proposed algorithm can only perform the model transfer for similar issues and fully retrain the model for different issues. In the future, we will combine the DDPG algorithm with other intelligent algorithms to improve the generality of the algorithmic model.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported in part by the Hubei Provincial Science and Technology Major Project of China (Grant No. 2020AEA011), in part by the National Ethnic Affairs Commission of the People's Republic of China (Training Program for Young and Middle-aged Talents) (No: MZR20007), in part by the National Natural Science Foundation of China (Grant No. 61902437), in part by the Hubei Provincial Natural Science Foundation of China (Grant No. 2020CFB629), in part by the Application Foundation Frontier Project of Wuhan Science and Technology Program (Grant No. 2020020601012267), in part by the Fundamental Research Funds for the Central Universities, South-Central MinZu University (No: CZQ21026), and in part by the Special Project on Regional Collaborative Innovation of Xinjiang Uygur Autonomous Region (Plan to Aid Xinjiang with Science and Technology) (2022E02035)

References

- [1] Y. Dong, E. Camci, E. Kayacan, Faster RRT-based nonholonomic path planning in 2D building environments using skeleton-constrained path biasing, *J. Intell. Rob. Syst.* 89 (3) (2018) 387–401.
- [2] U. Orozco-Rosas, O. Montiel, R. Sepúlveda, Mobile robot path planning using membrane evolutionary artificial potential field, *Appl. Soft Comput.* 77 (2019) 236–251.
- [3] C. Cai, S. Ferrari, Information-driven sensor path planning by approximate cell decomposition, *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 39 (3) (2009) 672–689.
- [4] T.P. Tunggal, A. Supriyanto, N.M.Z. Rochman, I. Faishal, I. Pambudi, I. Iswanto, Pursuit algorithm for robot trash can based on fuzzy-cell decomposition, *Int. J. Electr. Comput. Eng.* 6 (6) (2016) 2863.
- [5] Y. Xinyi, Z. Yichen, L. Liang, O. Linlin, Dynamic window with virtual goal (DW-VG): a new reactive obstacle avoidance approach based on motion prediction, *Robotica* 37 (8) (2019) 1438–1456.
- [6] J. Kennedy, R. Eberhart, Particle swarm optimization, in: *Proceedings of ICNN'95-international Conference on Neural Networks vol. 4*, IEEE, 1995, November, pp. 1942–1948.
- [7] X. Cai, Z. Hu, Z. Fan, A novel memetic algorithm based on invasive weed optimization and differential evolution for constrained optimization, *Soft Comput.* 17 (10) (2013) 1893–1910.
- [8] M. Clerc, J. Kennedy, The particle swarm - explosion, stability, and convergence in a multidimensional complex space, *IEEE Trans. Evol. Comput.* 6 (1) (2002) 58–73.
- [9] P. Sun, R. Shan, Predictive control with velocity observer for cushion robot based on PSO for path planning, *J. Syst. Sci. Complex.* 33 (4) (2020) 988–1011.
- [10] S. Shao, Y. Peng, C. He, Y. Du, Efficient path planning for UAV formation via comprehensively improved particle swarm optimization, *ISA Trans.* 97 (2020) 415–430.
- [11] Y. Dong, X. Zou, Mobile robot path planning based on improved DDPG reinforcement learning algorithm, in: *2020 IEEE 11th International Conference on Software Engineering and Service Science (ICSESS)*, IEEE, 2020, October, pp. 52–56.
- [12] G.T. Tu, J.G. Juang, Path planning and obstacle avoidance based on reinforcement learning for UAV application, in: *2021 International Conference on System Science and Engineering (ICSSE)*, IEEE, 2021, August, pp. 352–355.
- [13] E.H. Houssein, A.G. Gad, K. Hussain, P.N. Suganthan, Major advances in particle swarm optimization: theory, analysis, and application, *Swarm Evol. Comput.* 63 (2021) 100868.
- [14] Y. Shi, R. Eberhart, A modified particle swarm optimizer, in: *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No. 98TH8360)*, IEEE, 1998, May, pp. 69–73.
- [15] P.Y. Yang, F.I. Chou, J.T. Tsai, J.H. Chou, Adaptive-uniform-experimental-design-based fractional-order particle swarm optimizer with non-linear time-varying evolution, *Appl. Sci.* 9 (24) (2019) 5537.
- [16] C. Yang, W. Gao, N. Liu, C. Song, Low-discrepancy sequence initialized particle swarm optimization algorithm with high-order nonlinear time-varying inertia weight, *Appl. Soft Comput.* 29 (2015) 386–394.
- [17] M.S. Nobile, P. Cazzaniga, D. Besozzi, R. Colombo, G. Mauri, G. Pasi, Fuzzy Self-Tuning PSO: a settings-free algorithm for global optimization, *Swarm Evol. Comput.* 39 (2018) 70–85.
- [18] X. Ye, B. Chen, L. Jing, B. Zhang, Y. Liu, Multi-agent hybrid particle swarm optimization (MAHPSO) for wastewater treatment network planning, *J. Environ. Manag.* 234 (2019) 525–536.
- [19] R. Lan, L. Zhang, Z. Tang, Z. Liu, X. Luo, A hierarchical sorting swarm optimizer for large-scale optimization, *IEEE Access* 7 (2019) 40625–40635.
- [20] L.T. Al-Bahrani, J.C. Patra, A novel orthogonal PSO algorithm based on orthogonal diagonalization, *Swarm Evol. Comput.* 40 (2018) 1–23.
- [21] X. Xia, Y. Tang, B. Wei, L. Gui, Dynamic multi-swarm particle swarm optimization based on elite learning, *IEEE Access* 7 (2019) 184849–184865.
- [22] J. Ding, Q. Wang, Q. Zhang, Q. Ye, Y. Ma, A hybrid particle swarm optimization-cuckoo search algorithm and its engineering applications, *Math. Probl. Eng.* (2019) 1–12.
- [23] C.X. Yang, J. Zhang, M.S. Tong, A hybrid quantum-behaved particle swarm optimization algorithm for solving inverse scattering problems, *IEEE Trans. Antenn. Propag.* 69 (9) (2021) 5861–5869.
- [24] X. Xia, L. Gui, G. He, et al., An expanded particle swarm optimization based on multi-exemplar and forgetting ability, *Inf. Sci.* 508 (2020) 105–120.
- [25] X. Chen, H. Tianfield, C. Mei, W. Du, G. Liu, Biogeography-based learning particle swarm optimization, *Soft Comput.* 21 (24) (2017) 7519–7541.
- [26] H. Chen, C. Wang, J. Huang, J. Gong, Efficient use of heuristics for accelerating XCS-based policy learning in Markov games, *Swarm Evol. Comput.* 65 (2021) 100914.
- [27] T.P. Lillicrap, J.J. Hunt, A. Pritzel, et al., Continuous Control with Deep Reinforcement Learning, 2015 arXiv preprint arXiv:1509.02971.
- [28] L.J. Lin, Reinforcement Learning for Robots Using Neural Networks, Carnegie Mellon University (Ph. D. Thesis), 1992.
- [29] Y. Hou, L. Liu, Q. Wei, X. Xu, C. Chen, A novel DDPG method with prioritized experience replay, in: *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, IEEE, 2017, October, pp. 316–321.
- [30] J. Li, T. Yu, X. Zhang, F. Li, D. Lin, H. Zhu, Efficient experience replay based deep deterministic policy gradient for AGC dispatch in integrated energy system, *Appl. Energy* 285 (2021) 116386.
- [31] Z. Zhang, J. Chen, Z. Chen, W. Li, Asynchronous episodic deep deterministic policy gradient: toward continuous control in computationally complex environments, *IEEE Trans. Cybern.* 51 (2) (2019) 604–613.
- [32] T. de Bruin, J. Kober, K. Tuyls, R. Babuška, Improved deep reinforcement learning for robotics through distribution-based experience retention, in: *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2016, October, pp. 3947–3952.
- [33] T. Rojanoarpa, I. Kataeva, Density-based data pruning method for deep reinforcement learning, in: *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2016, December, pp. 266–271.
- [34] A.W. Moore, C.G. Atkeson, Prioritized sweeping: reinforcement learning with less data and less time, *Mach. Learn.* 13 (1) (1993) 103–130.
- [35] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized Experience Replay, 2015 arXiv preprint arXiv:1511.05952.
- [36] H. Xiang, J. Cheng, Q. Zhang, J. Liu, Synthesized prioritized data pruning based deep deterministic policy gradient algorithm improvement, in: *2018 IEEE International Conference on Information and Automation (ICIA)*, IEEE, 2018, August, pp. 121–126.
- [37] P. Li, X. Ding, H. Sun, S. Zhao, R. Cajo, Research on Dynamic Path Planning of Mobile Robot Based on Improved DDPG Algorithm, *Mobile Information Systems*, 2021.

- [38] X. Zhang, L. Duan, Energy-saving deployment algorithms of UAV swarm for sustainable wireless coverage, *IEEE Trans. Veh. Technol.* 69 (9) (2020) 10320–10335.
- [39] J.J. Liang, A.K. Qin, P.N. Suganthan, S. Baskar, Comprehensive learning particle swarm optimizer for global optimization of multimodal functions, *IEEE Trans. Evol. Comput.* 10 (3) (2006) 281–295.
- [40] B. Niu, H. Huang, L. Tan, Q. Duan, Symbiosis-based alternative learning multi-swarm particle swarm optimization, *IEEE ACM Trans. Comput. Biol. Bioinf* 14 (1) (2015) 4–14.
- [41] Y. Liu, W. Zhang, F. Chen, J. Li, Path planning based on improved deep deterministic policy gradient algorithm, in: 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), IEEE, 2019, March, pp. 295–299.
- [42] H. Xiang, J. Cheng, Q. Zhang, J. Liu, Synthesized prioritized data pruning based deep deterministic policy gradient algorithm improvement, in: 2018 IEEE International Conference on Information and Automation (ICIA), IEEE, 2018, August, pp. 121–126.